



Dense motion estimation of particle images via a convolutional neural network

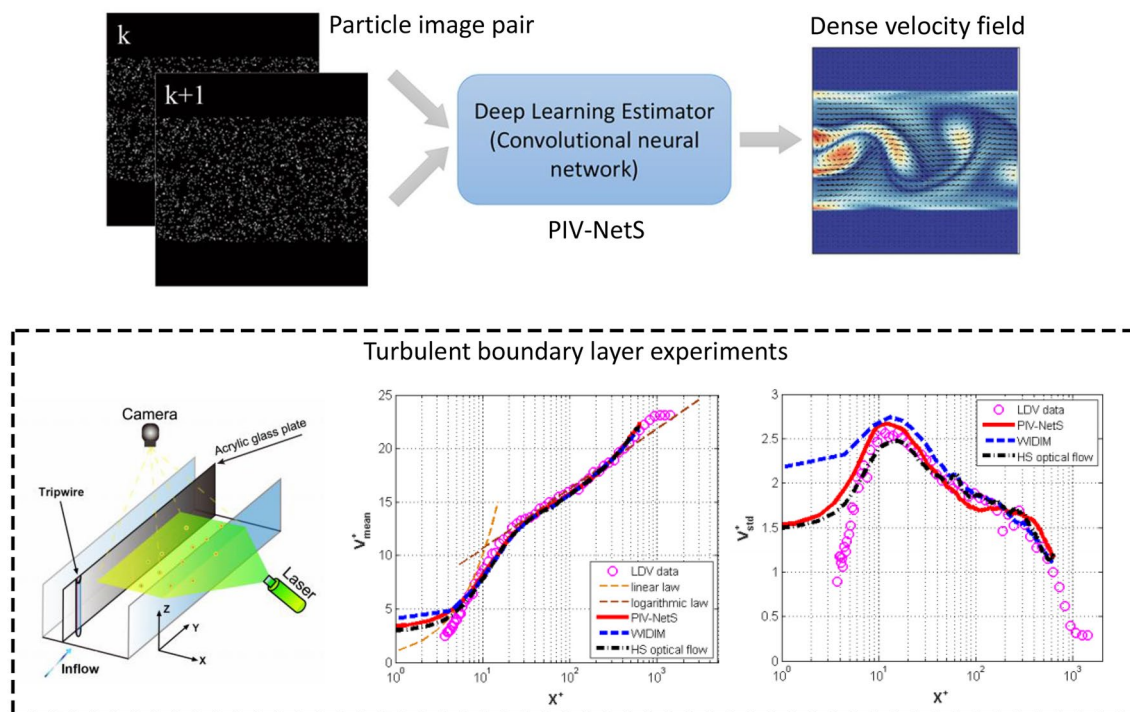
Shengze Cai¹ · Shichao Zhou² · Chao Xu¹ · Qi Gao³

Received: 25 November 2018 / Revised: 11 January 2019 / Accepted: 3 March 2019 / Published online: 22 March 2019
 © Springer-Verlag GmbH Germany, part of Springer Nature 2019

Abstract

In this paper, we propose a supervised learning strategy for the fluid motion estimation problem (i.e., extracting the velocity fields from particle images). The purpose of this work is to design a convolutional neural network (CNN) for estimating dense motion field for particle image velocimetry (PIV), which allows to improve the computational efficiency without reducing the accuracy. First, the network model is developed based on FlowNetS, which is recently proposed for end-to-end optical flow estimation in the computer vision community. The input of the network is a particle image pair and the output is a velocity field with displacement vectors at every pixel. Second, a synthetic dataset of fluid flow images is generated to train the CNN model. To our knowledge, this is the first time a CNN has been used as a global motion estimator for particle image velocimetry. Experimental evaluations indicate that the trained CNN model can provide satisfactory results in both artificial and laboratory PIV images. The proposed estimator is also applied to the experiment of turbulent boundary layer. In addition, the computational efficiency of the CNN estimator is much superior to those of the traditional cross-correlation and optical flow methods.

Graphical abstract



1 Introduction

Fluid visualization plays an increasingly important role in experimental fluid mechanics, as it helps the researchers to get a deeper insight into the complex flow phenomena. Among the flow visualization technologies, particle image velocimetry (PIV) (Raffel et al. 2007; Adrian and Westerweid 2011) is the most commonly used method, which can provide non-intrusive quantitative measurement of the velocity fields (e.g., planar PIV for two-dimensional measurement or tomographic PIV for three-dimensional measurement). One possible way to construct a PIV experiment is as follows. Small particles are first seeded into the flow medium. A laser sheet illuminates the flow domain to activate the particles, then successive images with fluorescent particles are recorded by a camera. Eventually, velocity vectors of the investigated flow are obtained by analyzing the consecutive images. Besides PIV hardware system development, how to estimate the motion fields from an image pair or sequence remains the key problem and main challenge in this area.

There are two main techniques for extraction of velocity fields from particle images: cross-correlation and optical flow methods. Correlation-based methods determine the displacement by searching the maximum of the cross-correlation between two interrogation windows of an image pair. The cross-correlation (namely the convolutional computation between two patches) can be generally computed by fast Fourier transform (FFT) (Wereley et al. 2002). Such an approach, which regards the displacement as the averaged velocity in a window, has been intensively studied and become well known in the community of experimental fluid mechanics. For example, Scarano (2002) presents a window deformation iterative multi-grid (termed WIDIM) method which enhances the precision and spatial resolution of numerical interrogation schemes. The WIDIM method allows a de-coupling between the spatial resolution and the dynamic range by using an iterative evaluation procedure with integrated window refinement, and it has been evaluated with good performance in the International PIV Challenges (Stanislas et al. 2005, 2008).

In recent years, global variational optical flow computation has become a promising technique to supplement traditional cross-correlation approaches. First proposed by Horn and Schunck (1981), optical flow has been developed greatly in the computer vision community, and great improvements have been presented such as Brox et al. (2004), Bruhn et al. (2005), Sun et al. (2010). These approaches usually rely on an objective functional consisting of a data term (based on brightness constancy assumption) and a regularization term. In contrast to the cross-correlation methods, optical flow can provide dense velocity fields (displacement vectors at every grid) by minimizing the cost function. Recently, variational

optical flow has attracted great interests from the experimentalists in fluid mechanics. The original Horn and Schunck (HS) formulation has been extended in various ways to cope with fluid motion estimation. For example, Corpetti et al. (2002, 2006) presents a fluid flow dedicated formulation based on the integrated continuity equation (ICE) and a second-order div-curl regularizer, that can preserve better the divergence and the vorticity of the flow. Liu and Shen (2008) suggests to use the projected motion equation and discusses exhaustively the relation between optical flow and fluid flow model. The review article (Heitz et al. 2010) gives a comprehensive discussion on different estimators associated with fluid flow. Moreover, optical flow formulation is also combined with turbulent models to improve the performance on estimating turbulent flow motion with small-scale vortices (Chen et al. 2015; Cai et al. 2018).

Comparisons between the optical flow and the cross-correlation methods in extracting velocity fields from particle images have been presented in Liu et al. (2015). It has been proven that both methods can be successfully applied to fluid motion estimation in most cases. Despite their differences, correlation-based and optical flow estimators both rely on some parameters being tuned (e.g., weighting coefficients for HS and the interrogation window size for WIDIM). Furthermore, a high computational cost is required for multi-grid or multi-resolution implementation, especially for the multi-resolution algorithm of HS method.

In this paper, we are applying a novel model—deep neural network—for particle image velocimetry which differs from correlation-based as well as optical flow methods. In the past decade, deep learning methods have been increasingly popular and applied to solve complicated problems such as classification or recognition. Especially, the convolutional neural networks (CNN) have been significantly successful in a number of computer vision tasks (Krizhevsky et al. 2012). The success of the CNN models has inspired us to develop deep learning technique for fluid motion estimation from particle images. In the community of experimental fluid dynamics, there are only a few works focusing on this topic up to now. One of them is presented in Lee et al. (2017), where a cascaded network architecture from Sun et al. (2013) is used with the parameters being trained by particle images. The philosophy of this network model (denoted by PIV-DCNN) is similar to the cross-correlation methods: the input of the model is a combination of two patches from the consecutive images in a size of 64×64 pixel and the output is a displacement vector with two components of the patch center. The performance of PIV-DCNN is competitive to the standard PIV methods. However, this kind of model employs 64×64 pixel window and it provides a sparse velocity field. Furthermore, it does not decrease the execution time in comparison with the standard PIV

methods even in GPU mode due to the repeated computation for a large number of patches.

In this work, we aim at investigating and improving the performance of using a deep network for particle image velocimetry. In contrary to Lee et al. (2017), we try to extract a high-resolution velocity field (one vector for one pixel) from images, rather than a sparse one, with very high efficiency. To achieve these, we develop the CNN model based on FlowNetS (Dosovitskiy et al. 2015), which is designed to receive two images as input and estimate the corresponding optical flow (in a full resolution) as output. According to Dosovitskiy et al. (2015), it is trained by a large synthetic FlyingChair dataset and achieve to provide acceptable accuracy of rigid-motion estimation. However, the original FlowNetS cannot be directly applied to fluid flow images (as we will show in the experimental section). The contributions of this paper include: on the one hand, we make several improvements for the architecture and settings of FlowNetS; on the other hand, we generate a synthetic dataset of particle images for CNN model training. These changes lead to a satisfactory CNN fluid motion estimator that can compete with traditional methods to some extent.

There are several advantages for using this kind of deep network for particle image velocimetry. Similar to the optical flow methods, the full CNN model provides a dense motion field. That means the result of this model contains more details (small-scale structures) of the flow. Moreover, although training the parameters of the model is time consuming, the training process can be done offline. Once the network has been well trained, the execution time of the estimation procedure is at the level of 10–100 ms for each image pair. Real-time estimation (high efficiency) with an acceptable accuracy can be achieved using this learning strategy. This could be promising in visual serving for active flow control in real time (Dao and Collewet 2012; Xu et al. 2008).

The article is organized as follows. The basic concept of the convolutional neural network is provided in Sect. 2. In Sect. 3, we describe the CNN network on the basis of the FlowNetS and the generation of synthetic training data. Experimental evaluations and results are demonstrated in Sects. 4 and 5, followed by the conclusions in Sect. 6.

2 Concept of convolutional neural network

Convolutional neural network (CNN), as one of the artificial neural networks (NN), has been widely applied as a powerful tool of computer vision technique (Krizhevsky et al. 2012; Chen et al. 2018). In general, a deep neural network consists of multiple layers (including input, output and hidden layers) and has a number of trainable parameters. The original information (e.g., an image) is fed into the CNN through the input layer, and the prediction (e.g., a classification result)

is given through the output layer. All the information is transferred and processed via the hidden layers inside the network. There are various hidden layers being designed. In this section, we only introduce layers that are applied in the neural network of this paper, including convolutional layer, pooling layer, activating layer and de-convolutional layer.

Convolutional layer The core building block of the CNN architecture is the so-called convolutional layer, where each neuron of output (y_j) is connected to only a fraction of input neurons (x_i) (see Fig. 1a). On the contrary, in a fully connected layer, there exists a connection between any output neuron (e.g., y_{j-1}) and input neuron, as indicated by crimson dashed arrows in Fig. 1a. Therefore, the forward computation of convolutional layer is in general more efficient and involves much fewer trainable parameters. In other words, if the relationship between input and output of one layer is expressed as $\mathbf{Y} = \mathbf{W}\mathbf{X} + \mathbf{b}$, where the input vector $\mathbf{X} = (x_1, x_2, \dots, x_i)^T$ and the output $\mathbf{Y} = (y_1, y_2, \dots, y_j)^T$, the weight matrix $\mathbf{W}$ is sparse for convolutional layers.

The local connectivity in Fig. 1a is inherently determined by the convolutional operation, which is illustrated in Fig. 1b with a simple example (here $b = 0$ for simplicity). There is a gray cube of size $3 \times 3 \times 4$, called filter or kernel, whose depth is always the same as the input (shown as blue cube in the size of $5 \times 5 \times 4$). The kernel moves in horizontal and vertical directions with certain stride (here $s = 1$), and multiplies the corresponding elements of input with its $3 \times 3 \times 4 = 36$ coefficients. Then, the sum of the 36 products gives the value of the output (cyan square) at the corresponding location. The depth of the output cube is equal to the number of kernels, which is 1 here. However, more kernels can be applied to extract more features from the input. In the convolutional network, kernels are the unknowns to be learned.

Other layers Besides the convolutional layers, a typical CNN architecture also contains the rectified linear unit (ReLU) layer and the pooling layers. The ReLU layer, representing the nonlinearity of the network, transfers the data through the ReLU function which can be formulated as $f_{\text{ReLU}}(x) = \max(0, x)$. The ReLU is widely applied in CNNs since the function and its gradient are both easily implemented. The pooling layer mainly plays a role of data down-sampling, which takes average or maximal values within a local region. Notice that although the pooling layers help to aggregate information in large areas and thus make the information processing more effective, they do reduce the resolution of input data. To refine the coarse representations and enable per pixel prediction, the de-convolutional (transposed-convolutional) layer is required. De-convolutional layer simply reverses the forward and backward computation of convolution and resembles an upsampling interpolation. The de-convolutional filters/kernels in such a layer does need not to be fixed (e.g., to bilinear upsampling), but

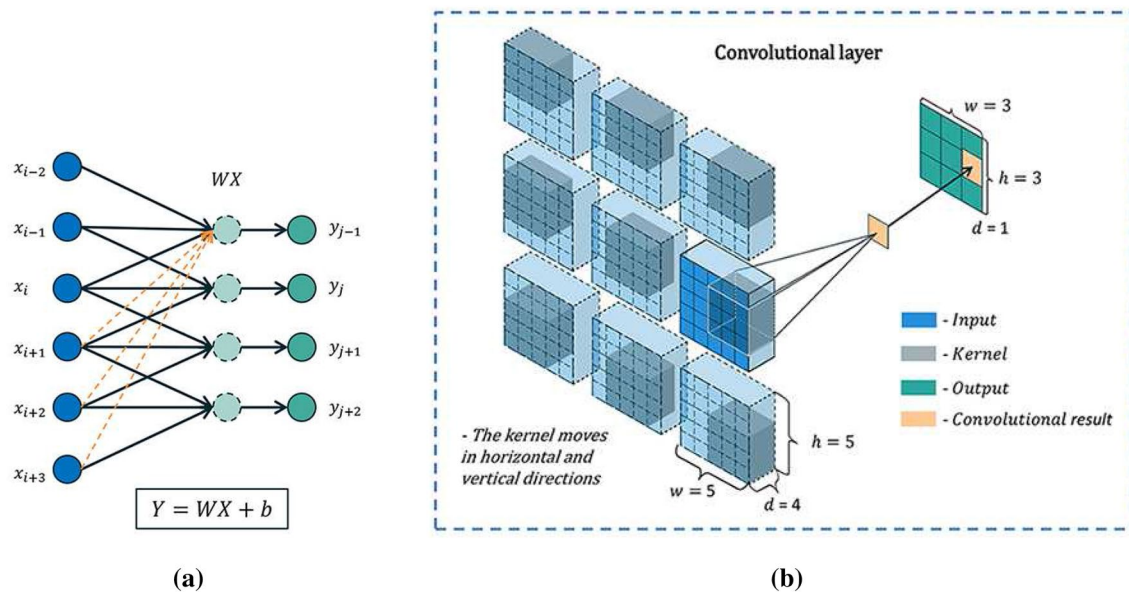


Fig. 1 A simple sketch of the convolutional layer. **a** Local connectivity between input and output. **b** Sketch of convolutional operation. The blue cube, cyan square and gray cube represent input, output and kernel, respectively

can be learned from training. Such layers are often applied especially in fully convolutional networks (Long et al. 2015).

Parameters optimization Once the CNN architecture is built up, the parameters of the network can be trained via minimizing a loss by gradient-based techniques. Since the training dataset is frequently huge, to speed up the training efficiency, special optimization algorithms (compared to those for classical parametric optimization problems) should be adopted. The algorithms, such as mini-batch gradient descent, momentum gradient descent (Qian 1999), RMSprop (Root mean square prop) (Tieleman and Hinton 2012), Adam (Adaptive moment estimation) (Kingma and Ba 2014) etc., have been widely applied and work well in a wide range of CNNs.

3 Convolutional neural network for PIV

In this section, we present the convolutional neural network for particle image velocimetry. This requires the introduction of FlowNetS at the beginning, since it is the fundamental architecture of the proposed model. Then, we also present how to generate the PIV dataset for CNN training.

3.1 Network structure and setting

In Dosovitskiy et al. (2015), two architectures of CNN for optical flow estimation are proposed: a generic architecture named FlowNetS and another one including a layer that correlates vectors of two input images named FlowNetC. These single networks are successively stacked together with

a warping process to form a complex structure, which is proposed in Ilg et al. (2017) and termed FlowNet2.0. According to the experiments in Ilg et al. (2017), FlowNet2.0 has become one of the state-of-the-art models on rigid-motion estimation problem. In this paper, to investigate the performance of a fully convolutional neural network in conducting the PIV estimation, we only apply one of the simple models, namely FlowNetS. The architecture of FlowNetS is demonstrated in Fig. 2 and a brief introduction is given as below.

As shown in Fig. 2, the six-channel blob composed of two three-channel images¹ It is fed into the network as input. The input then passes through the contracting part that compresses the spatial information into features with more channels. Each convolutional block of the contracting part consists of a convolutional layer which is strided with a factor of 2 (performs as a pooling layer as well) and a ReLU layer representing the nonlinearity. Subsequently, the expanding part of the network with de-convolutional layers refines the resolution to the desired output size. The output size of each de-convolutions increases by a factor of 2 again. There are also some skip connections (gray arrows in the figure) transferring blobs directly from a resolution level of contracting part to the corresponding level of expanding part. This can help preserve the image information at a higher spatial resolution.

¹ The original FlowNetS receives RGB images as input, hence six channels in all are required. However, PIV images are typically created using monochrome cameras. Therefore, two-channel input for this CNN is also sufficient.

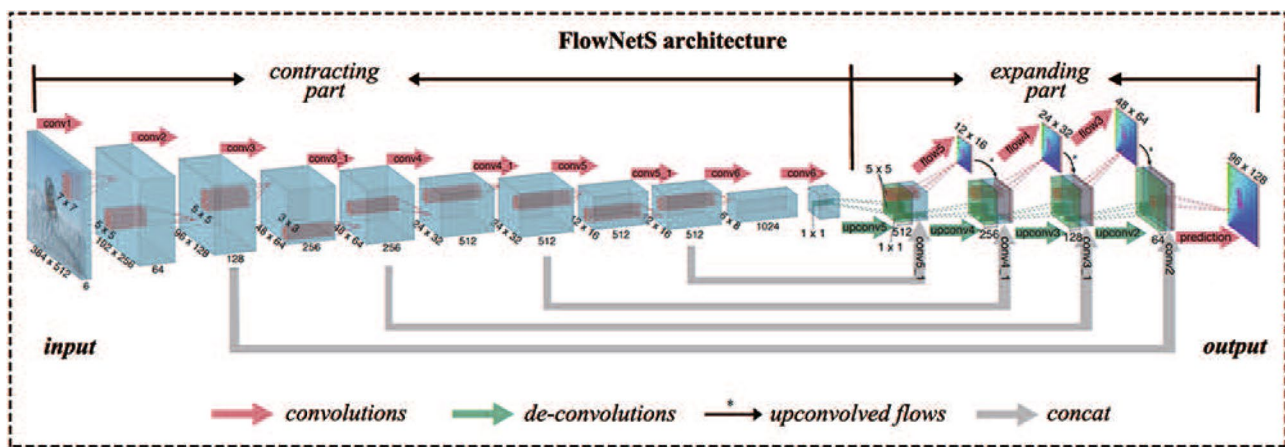


Fig. 2 FlowNetS architecture from Dosovitskiy et al. (2015). The network consists of contracting part and expanding part. The resolution of the final prediction is still four times smaller than the input. To

refine the estimation, the flow field is upsampled to full image resolution using interpolation methods. [This figure is modified from the paper Dosovitskiy et al. (2015)]

Eventually, the prediction of the network contains two channels, namely two velocity components of optical flow. However, the resolution of the output is still four times smaller than the input images. According to Dosovitskiy et al. (2015), further refinement (i.e., using more de-convolutional layers) does not improve the results compared to a bilinear upsampling method. Therefore, the flow field of the CNN output is refined to full image resolution using computationally less expensive interpolations.

As the training loss, FlowNetS uses the endpoint error (EPE) which is the Euclidean distance between the prediction and the ground truth, averaged over all pixels. Note that in the network, each expansion step produces an estimated flow field (i.e., flow5, flow4, flow3, prediction shown in the architecture). All these predicted results are considered in the loss function, thus the training loss function is composed of output errors from different layers with different weighting coefficients. For more details on FlowNetS, we refer to Dosovitskiy et al. (2015) and Mayer et al. (2018).

Above we have introduced the original structure and philosophy of FlowNetS. In this paper, several configurations of FlowNetS have been changed to improve the performance on PIV estimation.

- As mentioned before, the authors in Dosovitskiy et al. (2015) use interpolation methods to refine the flow field into full resolution instead of adding de-convolutional layers inside the network, since more de-convolutions cannot significantly improve the results. However, we found that it does not make sense for fluid motion estimation. According to our experiments, further refinement by network training (rather than by interpolation) does improve the performance on PIV estimation. One of the reasons may be that the fluid motion contains small-scale

(even sub-pixel) structures that may be ignored by interpolation methods. Therefore, we use a fully CNN with two more up-convolutional layers for PIV estimation.

- As we add two more de-convolutional layers at the last stage, two more terms (errors between outputs of the new layers and the ground truth motion) should be considered in the training loss function. Furthermore, the weighting coefficients of the objective function are reset. We greatly increase the weight at the last output level.
- Note that at each expanding step, the de-convolutional result (e.g., upconv2), the blob from the same resolution level of the contracting part (e.g., conv2), and the predicted flow field from the previous layer (e.g., flow3*) are concatenated along the depth axis. However, the data dimensionality of the image intensity and of the velocity vector are different, thus may influence the estimation results. Specifically, the magnitude of image intensity generally ranges from 0 to 255 (for an 8-bits image), while that of velocities ranges from 0 to 10 pixels in this work. To avoid this problem, magnitudes are normalized to 0–1 when data from different sources are processed together.

The employed network is trained using a stochastic optimization method—Adam (Kingma and Ba 2014). Following the original FlowNetS, we use a mini-batch of eight image pairs for each iteration process. The iterations start with learning rate 10^{-4} , then successively divide by 2 every 100K iterations after the first 300K. The whole training process requires over 1200K iterations.

3.2 PIV dataset generation

Training the neural networks by supervised strategies requires data with ground truth to optimize the model

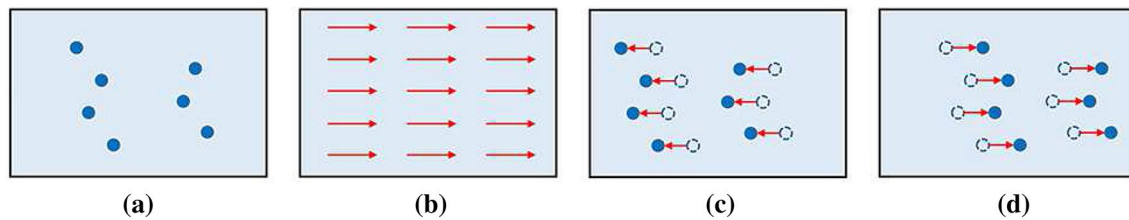


Fig. 3 The procedures of generating a PIV image pair. First, the particles are randomly seeded in an image domain, shown as blue dots in **a**. Next, the underlying flow motion is extracted from computational fluid dynamics simulation. For example, a uniform flow is displayed

in **b**. Eventually, the motion field is applied to the particles symmetrically (half displacement backward and forward) to obtain two particle images. The resulting images are illustrated in **c**, **d**

parameters. However, it is difficult to obtain the accurate velocity fields from experimental PIV data. We have to generate a synthetic dataset for CNN training. Following the general way to build a PIV dataset representing the fluid flow, we first generate a particle image as well as a flow motion pattern, then move the positions of the particles symmetrically by the flow motion to obtain an image pair. A sketch of the idea is illustrated in Fig. 3.

To generate the particle images, one can follow the public PIVlab,² particle image generator (PIG) in Raffel et al. (2007) or Lee et al. (2017). They are based on the idea that one particle can be described by a two-dimensional (2D) Gaussian function:

$$I(x, y) = I_0 \exp \left[\frac{-(x - x_0)^2 - (y - y_0)^2}{(1/8)d_p^2} \right], \quad (1)$$

where I_0 is the peak intensity in the Gaussian center, d_p denotes the particle diameter and (x_0, y_0) the position of the particle. Note that I_0 , d_p and (x_0, y_0) can be diverse from each particle. Furthermore, let us define the particle seeding density of the image as ρ (unit: particle per pixel, ppp), which is a global parameter for an image and affects the particle number in the observed domain. With specified values of I_0 , d_p , (x_0, y_0) and ρ , a typical particle image can be produced. Parameters for defining an image can be randomly selected in a proper range, as shown in Table 1. Figure 4 displays three examples of particle image produced by different parameters. All the images are in the resolution of 256×256 pixel.

As for the training motion, we extract the velocity fields from computational fluid dynamics (CFD). On the one hand, we simulate several flow benchmarks such as uniform flow (denoted by Uniform), backward stepping flow (Back-step) and the flow over a circular cylinder (Cylinder) with various configurations (e.g., Reynold number). Since these flows are

Table 1 The ranges of parameters for defining a synthetic particle image

Parameter	Range	Unit
Seeding density ρ	0.05–0.1	ppp
Particle diameter d_p	1–4	Pixel
Peak intensity I_0	200–255	Gray value

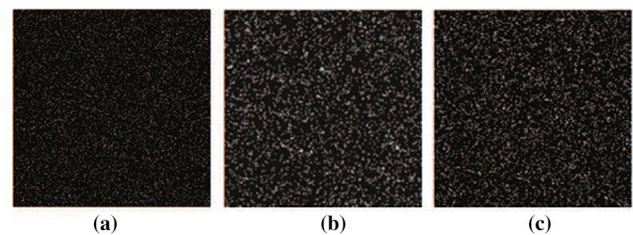


Fig. 4 Particle images generated by PIG with different parameters: **a** seeding density $\rho = 0.078$ ppp, particle diameter $d_p = 1.31$ pixel, **b** $\rho = 0.051$ ppp, $d_p = 3.68$ pixel, **c** $\rho = 0.069$ ppp, $d_p = 2.81$ pixel

unsteady flows (or unsteady during developing), the velocity fields of different time steps can be extracted. On the other hand, there are some online available CFD data that can also produce a number of flow structures, such as Carlier (2005), Li et al. (2008), Perlman et al. (2007) and Resseguier et al. (2017). Figure 5 demonstrates four of the flow motions that we use to generate the PIV dataset. The particle displacements of the whole dataset range from 0 to 10 pixels. We should note that these velocity fields used for generating particle image pairs are also the ground truth used for model training.

The combinations of randomly produced particle images and CFD flow motion patterns result in the whole PIV dataset. So far, we have obtained more than 13K particle image pairs with ground truth velocity fields. The composition of PIV dataset is presented in the “Appendix”. Moreover, to further increase the data diversity and avoid overfitting of neural networks, data augmentation is generally required. In FlowNetS, data augmentation is performed during the

² Available online: <https://github.com/mrahrault/PIVLab>.

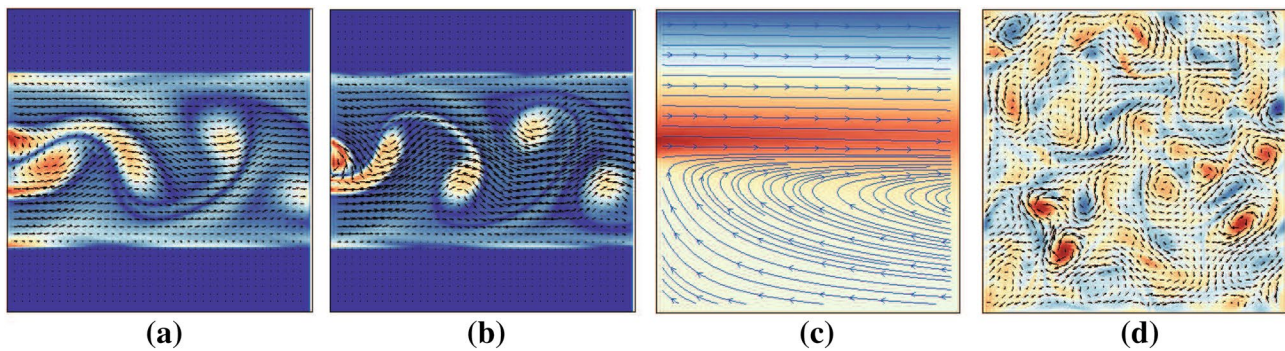


Fig. 5 Samples of some flows generated from CFD: **a** velocity field and vorticity map of the flow behind a circular cylinder ($Re = 150$); **b** velocity field and vorticity map of the flow behind a circular cylinder ($Re = 400$); **c** streamlines and vorticity map of the flow behind a backward step; **d** velocity field and vorticity map of a homogeneous and isotropic flow provided by Carlier (2005)

network training, which includes the following transformations: translation, rotation, scaling, changes of brightness and additive Gaussian noise. All these operations are processed in a specified range. For examples, as described in Dosovitskiy et al. (2015), the Gaussian noise has a sigma uniformly sampled from $[0, 0.04]$ and multiplicative brightness changes are sampled from $[0.5, 2]$.

The dataset is divided into two parts: most of the images are used for training, while a fraction of them is used for testing.

3.3 Notations of networks

In this work, the fundamental architecture of FlowNetS (Dosovitskiy et al. 2015) is employed. However, we not only generate a PIV dataset for model training but also change several configurations of the architecture to improve the performance on PIV estimation. These are the main contributions of this work.

For the purpose of distinguishing the changes, the proposed model in this paper is named by PIV-NetS hereafter. For comparison purpose, PIV-NetS lacking the last two de-convolutional layers (same architecture as FlowNetS) is termed PIV-NetS-noRef, which means the resolution of the prediction is not fully refined to the input image level during network computing. And FlowNetS refers to the original networks proposed in Dosovitskiy et al. (2015), that is trained by utilizing rigid-motion images (e.g., FlyingChairs dataset) rather than the PIV dataset.

3.4 General comments on networks

The main idea of this paper is illustrated in Fig. 6. Two particle images are fed into the CNN model and a dense velocity field is directly obtained. To use this estimator, one just set up an environment which can load the network (architecture and parameters). We indicate that it is advantageous to apply such a model as it estimates accurate and dense flow field

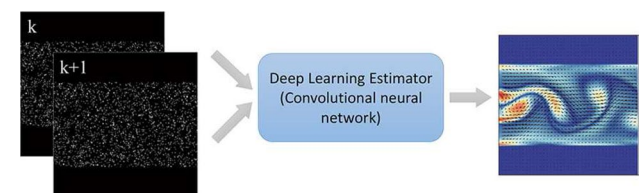


Fig. 6 Illumination of the main idea in this paper

with high efficiency. Note that there are some correlation-based methods which also provide the same advantages. For example, Champagnat et al. (2011) proposed an algorithm (termed FOLKI-PIV) for fast and accurate PIV using GPU acceleration on Lucas–Kanade approach. However, the principles are totally different from ours. First, the CNN model is learned from data. The parameters of the network are trained from the dataset introduced above. Once the CNN has been trained, it does not require a tuning process during the computation. Furthermore, the FOLKI-PIV is still based on window matching and the spatial resolution remains related to the window size. The CNN proposed in this paper is more like the variational optical flow methods. They perform better than the correlation-based methods with respect to the spatial resolution, as we can see in Sects. 4 and 5. Numerical assessments and experimental evaluations are described in the following sections, which show the effectiveness of the proposed deep learning estimator.

4 Evaluations on synthetic data

In this section, experimental results of motion estimation via the proposed CNN are presented. First, we compare different trained CNN models in Sect. 4.1 by evaluating the performance on our synthetic PIV dataset, to show that the enhanced configurations we make for FlowNetS are

effective. Next, the effect of several factors such as particle seeding density, Gaussian noise for CNN estimator is investigated in Sects. 4.2 and 4.3. Experiments on laboratory cases are carried out and discussed in the next section. Hereafter, metric for evaluation can be taken using root mean square error (RMSE) over N pixels of the image:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N [(u_i^t - u_i^e)^2 + (v_i^t - v_i^e)^2]} \quad (2)$$

where (u^t, v^t) and (u^e, v^e) denote the ground truth velocity and the estimated velocity, respectively. Furthermore, execution time, which is the superiority of CNN models, is also considered as an important criterion.

4.1 Results on training dataset

There are three CNN models we mentioned before, namely FlowNetS, PIV-NetS-noRef and PIV-NetS. The estimated errors of these models on our synthetic PIV dataset are shown in Table 2. As we can see from the numbers, original model which is trained from natural rigid-motion images (FlowNetS) performs badly on PIV estimation. PIV-NetS-noRef, which has the same network structure as FlowNetS but trained from particle images, provides much better

results than the previous. Furthermore, the PIV-NetS model with the modified architecture and configurations further improves the accuracy on PIV estimation by 30–50%. Such a learning model gives satisfactory results on not only the training data but also the testing set. We can also see that the PIV-NetS model performs perfectly on the simple cases (e.g., Uniform or Cylinder). However, the errors become larger on DNS turbulence and SQG cases, since these cases are much more complicated and contain a number of small-scale flow structures. To view more details, some examples of the estimated velocity field are demonstrated in Fig. 7. From the vorticity maps in the figure, one can also conclude that the proposed network model is superior to the other methods and has the ability to extract the flow motion from particle images.

4.2 Synthetic numerical assessments

To quantitatively demonstrate the performance of the proposed CNN estimators, we investigate the method on synthetic particle images with various parameters. The factors considered in this section include particle displacement, particle diameter, particle seeding density, particle out-of-plane motion and Gaussian noise addicted to images. The root mean square error is computed by taking the average of 100 tests for every investigated value. For comparison, we also demonstrate the results of the classical optical flow method—HS method (Horn and Schunck 1981) and the correlation-based window deformation iterative multi-grid [WIDIM, Scarano (2002)], since they are regarded as the baseline of PIV estimation. A multi-resolution algorithm (Heitz et al. 2010) with six pyramidal levels and two warping steps at each level is implemented to improve the performance of the variational optical flow computation. The HS method provides dense motion field as PIV-NetS (256×256 vectors), while the WIDIM method with step size of 8 pixel gives a 30×30 vector field. For the three-iteration WIDIM, unless particularly stated, the interrogation window size of the first iteration is 32×32 pixel and then reduces to 16×16 pixel at last.

The RMSE with respect to particle displacement is illustrated in Fig. 8a. This assessment is performed by evaluating the uniform flow which only contains constant horizontal movement. Images are simulated without noise and with parameters: seeding density $\rho = 0.1$ ppp, particle diameter $d_p = 2$ pixel. We can see that with the increase of particle displacement, the curves remain flat as a whole. The pixel-locking effect (Raffel et al. 2007) can be observed from the periodic error fluctuation due to the small particle diameter (here $d_p = 2$ pixel). To deal with large displacements, a starting window size of 64×64 is applied for the first iteration of the WIDIM method in this case. As a result, the multi-resolution optical flow performs the best regarding small

Table 2 Root mean square errors (unit: pixel) provided by different network models

Models	Uniform		Back-step	
	Train	Test	Train	Test
FlowNetS	0.899	0.816	1.443	1.459
PIV-NetS-noRef	0.123	0.126	0.136	0.139
PIV-NetS	0.058	0.059	0.069	0.072
Models	Cylinder		JHTDB-channel	
	Train	Test	Train	Test
FlowNetS	1.285	1.257	0.950	0.935
PIV-NetS-noRef	0.198	0.194	0.246	0.237
PIV-NetS	0.117	0.115	0.163	0.155
Models	DNS turbulence		SQG	
	Train	Test	Train	Test
FlowNetS	1.656	1.732	1.647	1.685
PIV-NetS-noRef	0.506	0.525	0.519	0.525
PIV-NetS	0.271	0.282	0.289	0.294

Descriptions of the datasets are given in the “Appendix”. Each data is divided into a training set and a testing set. Models are described in Sect. 3.3. Note again that FlowNetS is trained from natural rigid-motion images (trained parameters of the networks are provided by Ilg et al. (2017)) while PIV-NetS and PIV-NetS-noRef are trained from fluid images

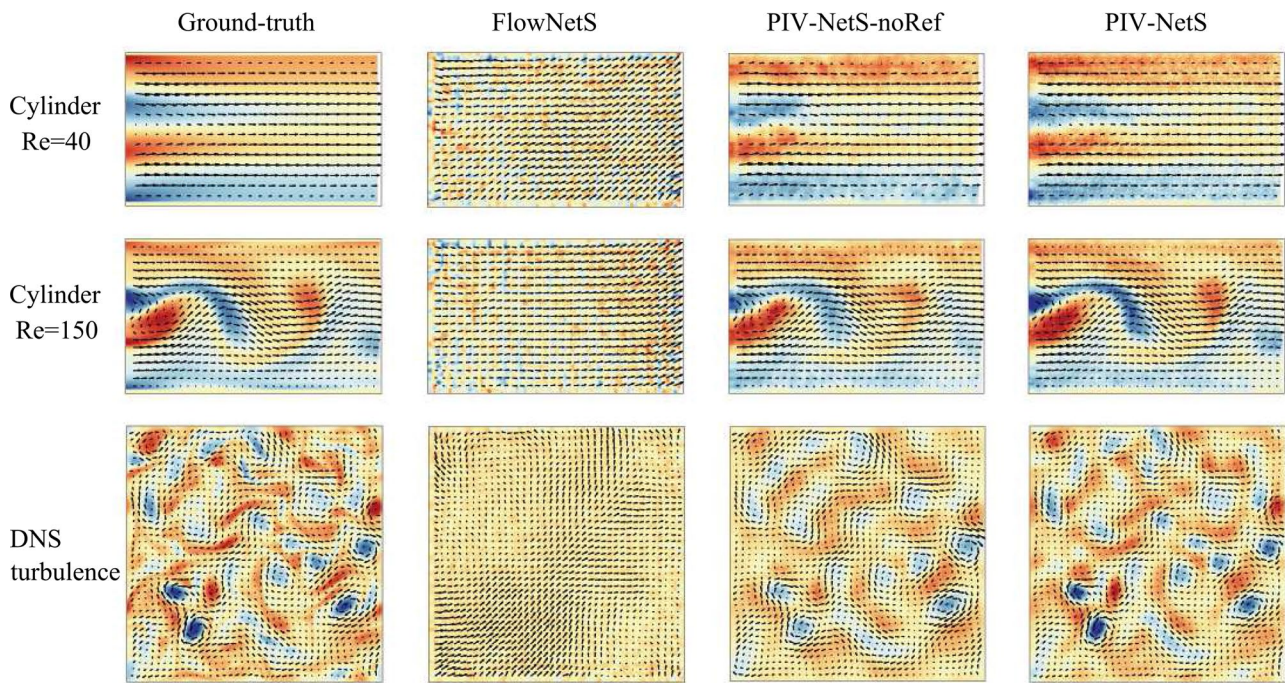


Fig. 7 The ground truth and the estimated fields extracted from particle images by different network models. The color map demonstrates the vorticity of the flows

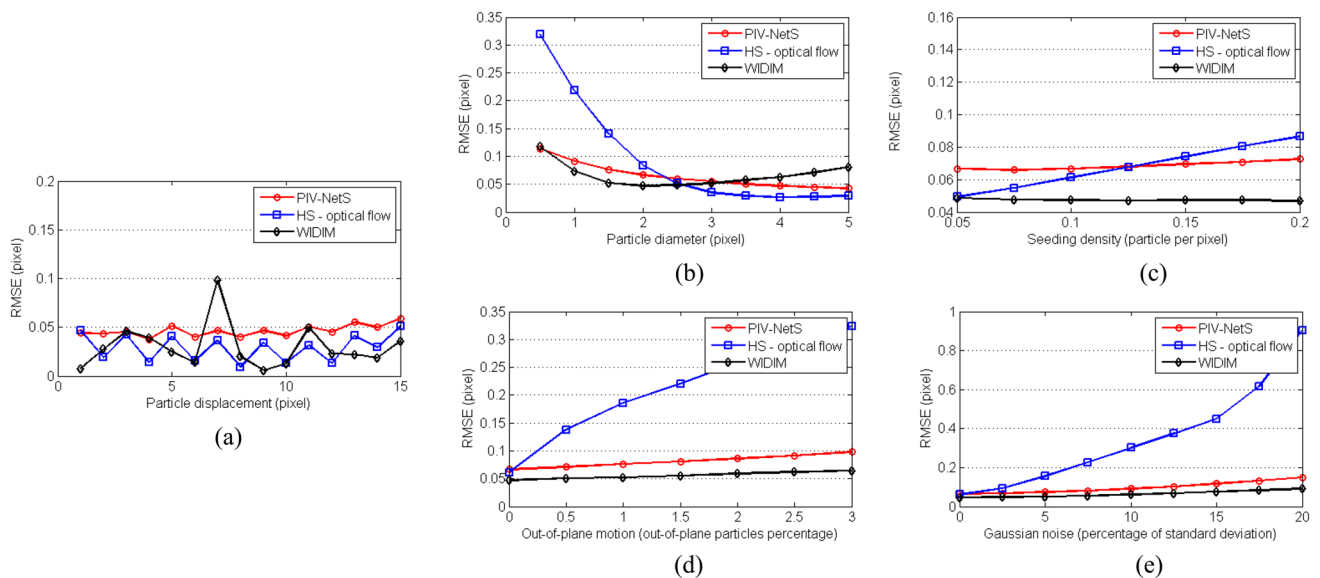


Fig. 8 RMSE with respect to **a** particle displacement of the uniform flow, **b** particle image diameter, **c** particle seeding density, **d** particle out-of-plane motion, **e** Gaussian noise of the vortex shedding flow over a circular cylinder

displacements, while the multi-pass WIDIM method outperforms the others when the displacement is larger than 12 pixel. The PIV-NetS model provides slightly larger errors. However, we can see that the CNN model is robust against the particle displacement. Note that the WIDIM method only estimates sparse velocity field with 30×30 vectors.

The RMSE with respect to particle image diameter is illustrated in Fig. 8b, which is evaluated on the vortex shedding flow over a circular cylinder (maximum particle displacement is about 3 pixel). Images are simulated without noise and with seeding density $\rho = 0.1$ ppp. In general, the HS method outperforms the others according to the curves.

The PIV-NetS performs better than the WIDIM method when the particle diameter is larger than 3 pixels. The error of PIV-NetS decreases when the particle diameter increases.

As for another important factor of particle images, the particle seeding density is investigated in Fig. 8c. The RMSE of PIV-NetS gradually increases when the particle concentration increases with a fixed particle diameter $d_p = 2$ pixel. In this case, the estimations of the HS method and the WIDIM method are more accurate. As we can see from the figure, the influence of the seeding density is relatively small for all these estimators.

The error regarding the out-of-plane motion is provided in Fig. 8d. The out-of-plane motion is represented by the ratio between the number of out-of-plane particles and the number of all particles. Once again, images are simulated under parameters: seeding density $\rho = 0.1$ ppp and particle diameter $d_p = 2$ pixel. It is obviously seen that the RMSEs of the WIDIM and PIV-NetS are directly and slightly proportional to the rate of out-of-plane motion. On the contrary, the error given by the HS method increases drastically with respect to the out-of-plane motion.

Furthermore, the effect of image noise is investigated in Fig. 8e. Gaussian white noise with different variance, which is set as a percentage of the standard deviation of the synthetic images, is added to every pixel of the synthetic images. As shown in the figure, for the CNN model, the error increases slightly when the variance becomes larger. The WIDIM approach provides similar trend to the PIV-NetS with respect to the Gaussian white noise, while the estimation of WIDIM is slightly better than that of the PIV-NetS model. On the other hand, the estimation results of HS method are quite sensitive to the noise. The HS method only outperforms the PIV-NetS model at low noise levels.

According to the assessments discussed above, we find that the motion estimation result of PIV-NetS is slightly worse than that of the multi-resolution HS method and the WIDIM method under the ideal conditions. However, there are several main advantages of this CNN estimator. First, as we can see from the figures, the CNN model is relatively robust to the noises (Gaussian white noise or out-of-plane motion), while the HS method is quite sensitive. This is reasonable since the noises are also taken into account in the training dataset for CNN model, while the HS method relies on the strict brightness constancy assumption. As assessed in the third PIV Challenge Case B: the classical optical flow methods is more suitable to be applied to the high-quality particle images and cannot process the images well with small signal-noise-ratio (SNR). Second, the computational efficiency of CNN estimator, which is not demonstrated in the figures, is superior to the variational optical flow method as well as the three-pass WIDIM methods. The execution time of the PIV-NetS model for one image pair (256×256 pixel) is around 40

ms under the GPU mode. The corresponding value converted into the CPU mode with the same computer is no more than 150 ms. By contrary, about 1000 ms and 400 ms of CPU computation is required for the HS and WIDIM method, respectively, since the multi-resolution or multi-grid algorithm are generally time consuming. Moreover, we should outline that the HS method and the PIV-NetS model both provide dense motion field, while the WIDIM method gives sparse estimation. The WIDIM method can provide better estimations in most simple cases. However, as we can see in the next experiment (Sect. 4.3), the PIV-NetS can extract more small-scale vortices compared to the WIDIM method.

4.3 Test on spatial resolution

The effective spatial resolution of the PIV estimators can be assessed using vortical flows with varying wavelength. We generate the velocity field as follows:

$$\begin{aligned} u(i, j) &= 3 \cos \left[\frac{2\pi i}{250 - 0.3i} \right] \sin \left[\frac{2\pi j}{250 - 0.3j} \right], \\ v(i, j) &= 3 \sin \left[\frac{2\pi i}{250 - 0.3i} \right] \cos \left[\frac{2\pi j}{250 - 0.3j} \right], \end{aligned} \quad (3)$$

where i, j denote the index of the image pixel along x - and y -direction, respectively. According to this expression, the wavelength (i.e., the size of vortex) decreases from the top-left corner to the bottom-right corner. The magnitude of v -component velocity and the true flow vectors is illustrated in Figs. 9a and 10a, respectively.

The remainder of Figs. 9 and 10 shows the comparison between the results estimated by different methods. We compare the CNN model (PIV-NetS) with the multi-resolution HS method and the WIDIM method, which are described in the previous section. The HS method with smoothing coefficient $\lambda = 100$ provides dense motion field as PIV-NetS (512×512 vectors). The WIDIM method employs interrogation window of 32×32 pixel and step size of 4 pixels, thus gives a vector field of 120×120 .

As shown in Figs. 9b and 10b, the WIDIM method is unable to estimate the small-scale vortices, that results in large error at the bottom-right corner. In Figs. 9c and 10c, we can see that the small-scale components can be resolved very well by the HS method. The estimated vortical flow of the HS method is quite consistent with the ground truth. Results of the proposed CNN model are demonstrated in Figs. 9d and 10d. Compared to the HS method, the PIV-NetS can also resolve well the small-scale vortices while it gives worse estimation at the bottom-left corner. The proposed method can recover most of the vortices in this test. However, more training data related to the flow patterns at the bottom-left

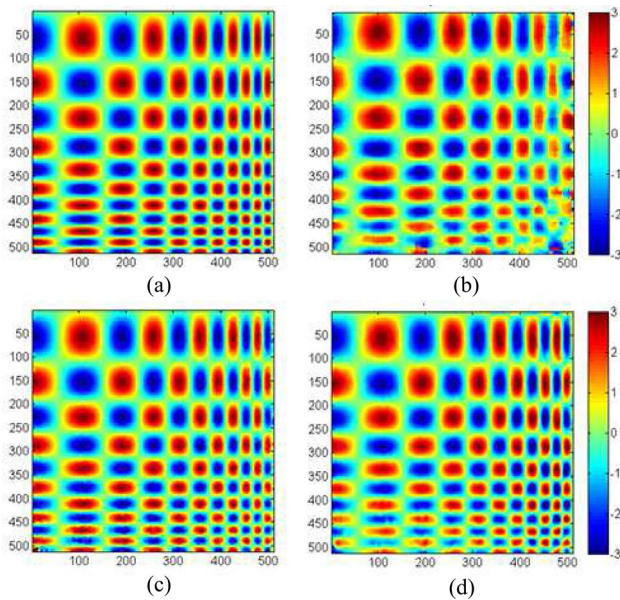


Fig. 9 Test on spatial resolution. The background color maps represent the magnitudes of v -component velocity from **a** the ground truth, **b** the WIDIM method, **c** the HS optical flow method, **d** the proposed PIV-NetS model

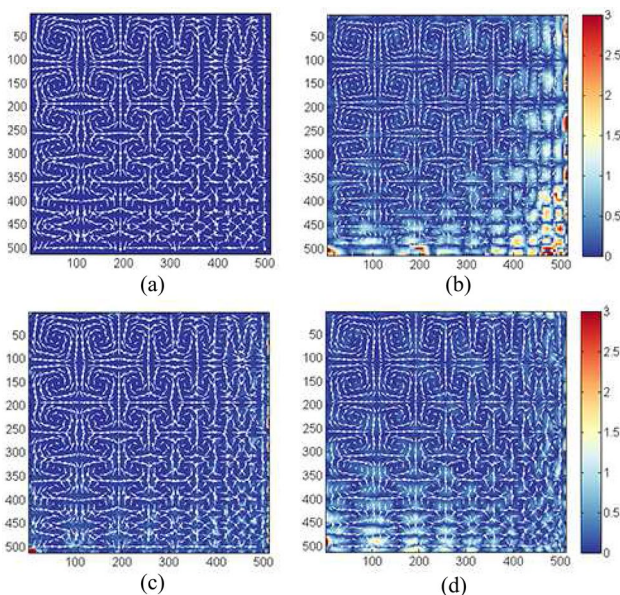


Fig. 10 Test on spatial resolution. Velocity fields and errors of **a** the ground truth, **b** the WIDIM method, **c** the HS optical flow method, **d** the proposed PIV-NetS model. The background color map represents the absolute error of v -component particle displacement

corner could be considered. Once again, we outline that the CNN model cannot outperform the HS method with respect to the accuracy while it is more efficient.

5 Evaluations on experimental data

5.1 Test on vortex pair flow and jet flow

For qualitative demonstration, particle images of a vortex pair flow and a jet flow are considered at this moment. Besides the HS approach and the WIDIM method, here we also compare the PIV-NetS with the PIV-DCNN model (Lee et al. 2017), which is another kind of convolutional neural network for PIV estimation. The HS method uses six pyramidal levels here and a large weighting coefficient $\lambda = 2000$. The WIDIM method employs interrogation window of 32×32 pixel and step size of 16 pixels, while the PIV-DCNN model uses 64×64 pixel window with the same step size. Results are shown in Fig. 11. It seems that the estimation of PIV-NetS is reasonable for the flows. Note that the PIV-NetS and HS models give dense flow field so that we can see more details from the maps. However, the result of HS method is not smooth enough probably due to the noise. The computation time of different approaches is given in Table 3. The proposed CNN model shows superiority compared to others with respect to efficiency. The execution time of PIV-NetS is much faster than that of PIV-DCNN with GPU acceleration, since the PIV-DCNN model requires repeated computation for a large number of patches. This experiment primarily proves that the CNN model trained from synthetic data also adapts to those real experimental images, which are not included in the training process.

5.2 Test on turbulent boundary layer flow

To investigate the performance on real experiments, the CNN model is applied to estimate the motion field of the near-wall flow for turbulent boundary layer. The experimental setup is illustrated in Fig. 12, which is conducted in a water tunnel. A flat acrylic glass plate is mounted in the tunnel to generate the turbulent boundary layer. The distance between the plate and the sidewall is large enough for the fully developed boundary layer. The turbulent flow is produced by an attached tripwire near the entrance of the inflow. As shown in the figure, the observed plane is parallel to the bottom of the tunnel. Particle images (2058×2456 pixel, 16 bits) are captured by a high-speed camera. Based on the camera settings, a magnification of 17.4 pixels/mm is obtained. In this case, the maximum displacement of the measured particle is around 12 pixels.

To validate the estimations, the basic properties of the turbulent boundary layer are measured by laser Doppler velocimetry (LDV). For this experiment, the skin friction velocity u_τ is approximately 5.3 mm/s, while the free stream-wise velocity u_∞ is about 123.4 mm/s. The corresponding Reynolds number based on the skin friction velocity (Re_τ)

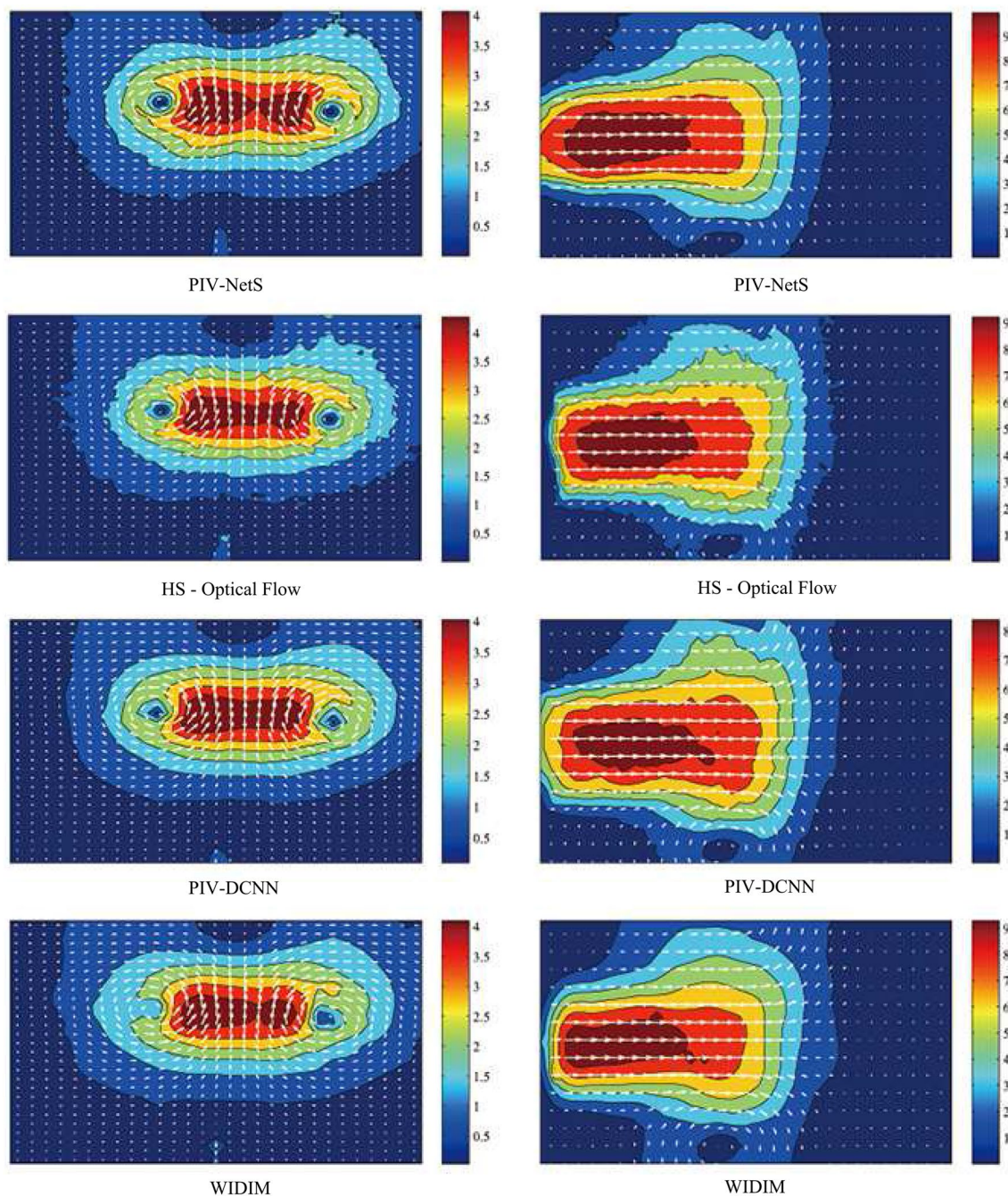


Fig. 11 Velocity fields estimated from laboratory PIV images: a vortex pair flow (left) and a jet flow (right). The color background with contours denotes the corresponding velocity magnitude

Table 3 Execution time (units: ms) of different methods for laboratory PIV estimation

	PIV-NetS	HS	WIDIM	PIV-DCNN	
	GPU	CPU	CPU	CPU	GPU
Vortex pair flow	51 (480×512)	3171 (480×512)	916 (27×29)	6970 (27×29)	2388 (27×29)
Jet flow	79 (512×1024)	8140 (512×1024)	1713 (29×61)	15,328 (29×61)	4721 (29×61)

The numbers in the brackets represent the valid vectors of the estimated fields

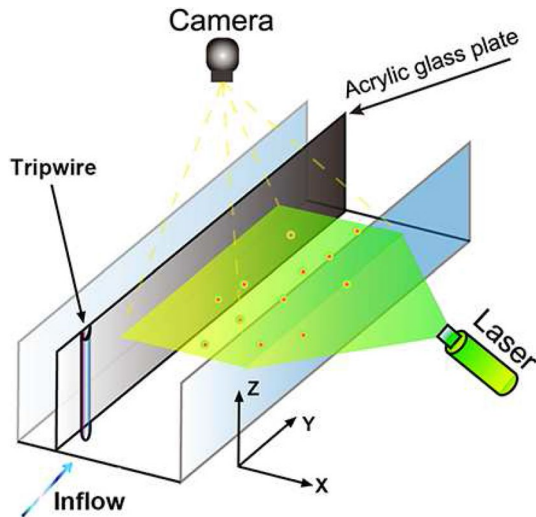


Fig. 12 Experimental setup of the turbulent boundary layer

is 1040. Let X^+ denotes the normalized distance between the measured point and the acrylic glass plate, and V^+ the normalized velocity component which is parallel to the plate. The mean velocity profile V_{mean}^+ and the fluctuating velocity profile V_{std}^+ with respect to X^+ is assessed in this experiment. For comparison, we demonstrate the results of the PIV-NetS model, the WIDIM and the HS optical flow methods along with the LDV data. Likewise, the three-pass WIDIM method is performed with a window size of 32×32 pixel and step size of 16 pixels, while the multi-resolution HS approach is set with six pyramidal levels. The estimated velocity fields are processed by a median filter.

The mean velocity profile and the fluctuating velocity profile estimated by different methods are illustrated in Fig. 13. The x -markers represent the results of the LDV, which is considered as the reference in this experiment. In the near-wall region, namely $X^+ < 10$, obvious differences among those lines can be observed in the figures. Specifically, the curves of PIV-NetS and HS method are much closer to the LDV data than that of the correlation-based method (WIDIM). The results of HS method is slightly lower than that of the PIV-NetS. This indicates that the variational optical flow method performs the best for the small-scale flow structures near the wall, followed by the PIV-NetS and then WIDIM. In the logarithmic region, the mean velocity profiles of the investigated approaches are all close to the LDV data and satisfied to the logarithmic law, while the fluctuating profiles appear to be different. The velocity fluctuation of the WIDIM is higher than the others including the LDV data. The fluctuation of PIV-NetS is higher than that of the optical flow method at first and then become lower. Overall, the HS method is closest to the results of LDV, which shows that the HS method is superior to the others for the estimation of small-scale components.

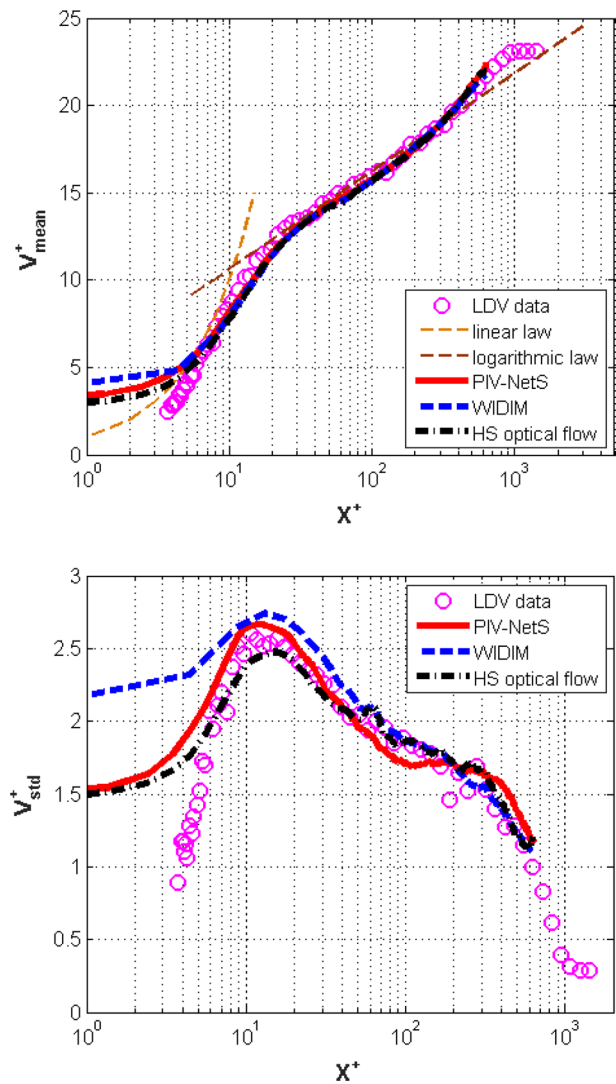


Fig. 13 The mean velocity profile (top) and the fluctuating velocity profile (bottom) of the turbulent boundary layer estimated by different methods

However, we should outline again that the computational time of HS method is much longer than those of the other two methods. As a new technology for PIV estimation, the deep learning estimator PIV-NetS is proved to be effective for real experimental particle images based on this assessment of turbulent boundary layer.

6 Conclusions and future works

In this paper, we propose a deep learning method for particle image velocimetry. The fully convolutional neural network is developed from FlowNetS (Dosovitskiy et al. 2015). The input of the network is a particle image pair and the output is a velocity field with displacement vectors at every pixel.

Since the original FlowNetS lacks ability to estimate the fluid motion from particle images (as shown in Fig. 7), we make several improvements in the model: (a) two additional de-convolutional layers are introduced to refine the resolution so that more details can be extracted; (b) settings of the loss function for network training is updated; (c) a normalization step is required to balance the values of image intensity and of velocity fields.

The proposed CNN model is trained using a synthetic PIV dataset described in Sect. 3.2. We do a number of numerical assessments to demonstrate the performance of the proposed CNN estimators. The influence from various parameters of the particle images, including particle seeding density, additive Gaussian noise, etc, are investigated. Experimental evaluation indicates that the end-to-end CNN model can estimate dense velocity fields with satisfactory accuracy for both artificial and laboratory particle images. Most importantly, the trained CNN model verifies great advantage of efficiency. High processing efficiency of this model makes it possible to achieve real-time estimation for particle image velocimetry. In this paper, although the PIV-NetS is trained from a synthetic dataset, it provides competitive estimation results for real experimental data, including the vortex pair flow, jet flow and the turbulent boundary layer. The proposed estimator can be applied to various applications. We can expect that once the CNN model has been well trained, it does not need to be re-trained for a change of imaging setup or flow. The offline training is not a procedure which is frequently required.

The attempt in this paper shows that deep learning can be a promising technique in conducting PIV estimation, and competes with traditional cross-correlation and optical flow methods. However, we believe there is tremendous room for improvement on this topic. First, there are some other networks proposed in the computer vision community for optical flow computation. For example, it is worth trying to evaluate the performance of FlowNet2.0 (Ilg et al. 2017), which is the successor of FlowNetS, trained by PIV dataset. Using other deep learning models as the basic could improve the estimation accuracy. However, the architecture of FlowNet2.0 is much more complicated than FlowNetS so that the size of model parameters and computational time should be larger. Moreover, for deep learning strategy, training data is an important factor. Although in this paper we have generated a number of data (particle images with ground truth motion), we think the amount and variety of PIV dataset should be increased. Specifically, data with noises should be further taken into account while training the CNN model, to work better on the real experimental images. The possible directions mentioned here are being investigated in the near future.

Acknowledgements This work was supported in part by the National Natural Science Foundation of China under Grant 61473253, the Foundation for Innovative Research Groups of the National Natural Science Foundation of China under Grant 61621002 and the Fundamental Research Funds for the Central Universities.

A data sets of particle images for training

As introduced in Sect. 3.2, in order to train the neural network, we generate a PIV dataset with more than 13 thousand items (where each item consists of an image pair and a ground truth motion field). The dataset includes various flow motion structures generated by computational fluid dynamics (CFD). Specifically, cases such as Uniform, Back-step, Cylinder are simulated by our own. DNS turbulence and surface quasi-geostrophic (SQG) cases are provided in Carlier (2005) and Resseguier et al. (2017), respectively. Finally, the Johns Hopkins Turbulence Databases can be found in Li et al. (2008), Perlman et al. (2007). Details of the dataset are given in Table 4. The data as well as the trained PIV-NetS model can be requested from the authors and will be accessed online soon.

Table 4 Data sets of particle images for neural network training

Case name	Description and main parameter	Number of image pair
Uniform	Uniform flow in the whole domain	1000
Back-step	Backward stepping flow $Re = 800$	600
	Backward stepping flow $Re = 1000$	600
	Backward stepping flow $Re = 1200$	1000
	Backward stepping flow $Re = 1500$	1000
Cylinder	Flow over a circular cylinder $Re = 40$	50
	Flow over a circular cylinder $Re = 150$	50
	Flow over a circular cylinder $Re = 200$	500
	Flow over a circular cylinder $Re = 300$	500
	Flow over a circular cylinder $Re = 400$	500
DNS turbulence	A homogeneous and isotropic flow	2000
SQG	Sea surface flow driven by a Surface	1500
	Quasi-geostrophic model	
JHTDB-channel	Forced isotropic turbulence provided by	1600
	Johns Hopkins turbulence databases	
JHTDB-mhd1024	Forced MHD turbulence provided by	800
	Johns Hopkins turbulence databases	
JHTDB-iso-tropic1024	Channel flow provided by	2000
	Johns Hopkins turbulence databases	

References

- Adrian R, Westerweel J (2011) Particle image velocimetry. Cambridge University Press, Cambridge
- Brox T, Bruhn A, Papenberger N, Weickert J (2004) High accuracy optical flow estimation based on a theory for warping. In: Proceedings of the European conference on computer vision, pp 25–36
- Bruhn A, Weickert J, Schnörr C (2005) Lucas/Kanade meets Horn/Schunck: combining local and global optic flow methods. *Int J Comput Vis* 61(3):211–231
- Cai S, Mémin E, Dérian P, Xu C (2018) Motion estimation under location uncertainty for turbulent fluid flows. *Exp Fluids* 59(1):8
- Carlier J (2005) Second set of fluid mechanics image sequences. European project fluid image analysis and description (FLUID). <http://www.fluid.irisa.fr>
- Champagnat F, Plyer A, Le Besnerais G, Leclaire B, Davoust S, Le Sant Y (2011) Fast and accurate PIV computation using highly parallel iterative correlation maximization. *Exp Fluids* 50(4):1169–1182
- Chen X, Zillé P, Shao L, Corpetti T (2015) Optical flow for incompressible turbulence motion estimation. *Exp Fluids* 56(1):1–14
- Chen L, Papandreou G, Kokkinos I, Murphy K, Yuille A (2018) Deeplab: semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected CRFs. *IEEE Trans Pattern Anal Mach Intell* 40(4):834–848
- Corpetti T, Mémin E, Pérez P (2002) Dense estimation of fluid flows. *IEEE Trans Pattern Anal Mach Intell* 24(3):365–380
- Corpetti T, Heitz D, Arroyo G, Mémin E, Santa-Cruz A (2006) Fluid experimental flow estimation based on an optical-flow scheme. *Exp Fluids* 40(1):80–97
- Dao X, Collewet C (2012) Drag reduction of the plane poiseuille flow by partitioned visual servo control. In: Proceedings of the American control conference, pp 4084–4089
- Dosovitskiy A, Fischer P, Ilg E, Hausser P, Hazirbas C, Golkov V, Van der Smagt P, Cremers D, Brox T (2015) FlowNet: learning optical flow with convolutional networks. In: Proceedings of the IEEE international conference on computer vision, pp 2758–2766
- Heitz D, Mémin E, Schnörr C (2010) Variational fluid flow measurements from image sequences: synopsis and perspectives. *Exp Fluids* 48(3):369–393
- Horn B, Schunck B (1981) Determining optical flow. *Artif Intell* 17(1–3):185–203
- Ilg E, Mayer N, Saikia T, Keuper M, Dosovitskiy A, Brox T (2017) FlowNet 2.0: evolution of optical flow estimation with deep networks. In: Proceedings of the IEEE conference on computer vision and pattern recognition, vol 2
- Kingma D, Ba J (2014) Adam: a method for stochastic optimization. In: Proceedings of the 3rd international conference on learning representations (ICLR)
- Krizhevsky A, Sutskever I, Hinton G (2012) Imagenet classification with deep convolutional neural networks. *Adv Neural Inf Process Syst* 25(2):1097–1105
- Lee Y, Yang H, Yin Z (2017) PIV-DCNN: cascaded deep convolutional neural networks for particle image velocimetry. *Exp Fluids* 58(12):171
- Li Y, Perlman E, Wan M, Yang Y, Meneveau C, Burns R, Chen S, Szalay A, Eyink G (2008) A public turbulence database cluster and applications to study Lagrangian evolution of velocity increments in turbulence. *J Turbul* 9(31):1–29. <https://doi.org/10.1080/14685240802376389>
- Liu T, Shen L (2008) Fluid flow and optical flow. *J Fluid Mech* 614:253–291
- Liu T, Merat A, Makhmalbaf H, Fajardo C, Merati P (2015) Comparison between optical flow and cross-correlation methods for extraction of velocity fields from particle images. *Exp Fluids* 56(8):166
- Long J, Shelhamer E, Darrell T (2015) Fully convolutional networks for semantic segmentation. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp 3431–3440
- Mayer N, Ilg E, Fischer P, Hazirbas C, Cremers D, Dosovitskiy A, Brox T (2018) What makes good synthetic training data for learning disparity and optical flow estimation? *Int J Comput Vis* 1:1–19
- Perlman E, Burns R, Li Y, Meneveau C (2007) Data exploration of turbulence simulations using a database cluster. In: Proceedings of the 2007 ACM/IEEE conference on supercomputing
- Qian N (1999) On the momentum term in gradient descent learning algorithms. *Neural Netw* 12(1):145–151
- Raffel M, Willert C, Wereley S, Kompenhans J (2007) Particle image velocimetry: a practical guide. Springer, Berlin
- Resseguier V, Mémin E, Chapron B (2017) Geophysical flows under location uncertainty, part II: quasigeostrophic models and efficient ensemble spreading. *Geophys Astrophys Fluid Dyn* 111(3):177–208
- Scarano F (2002) Iterative image deformation methods in PIV. *Meas Sci Technol* 13(13):R1
- Stanislas M, Okamoto K, Kähler CJ, Westerweel J (2005) Main results of the second international PIV challenge. *Exp Fluids* 39(2):170–191
- Stanislas M, Okamoto K, Kähler CJ, Westerweel J, Scarano F (2008) Main results of the third international PIV challenge. *Exp Fluids* 45(1):27–71
- Sun D, Roth S, Black M (2010) Secrets of optical flow estimation and their principles. In: Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR), pp 2432–2439
- Sun Y, Wang X, Tang X (2013) Deep convolutional network cascade for facial point detection. In: Proceedings of IEEE conference on computer vision and pattern recognition, pp 3476–3483
- Tieleman T, Hinton G (2012) Lecture 6.5-rmsprop: divide the gradient by a running average of its recent magnitude. COURSE: *Neural Netw Mach Learn* 4(2):26–31
- Wereley S, Gui L, Meinhart C (2002) Advanced algorithms for micro-scale particle image velocimetry. *AIAA J* 40(40):1047–1055
- Xu C, Schuster E, Vazquez R, Krstic M (2008) Stabilization of linearized 2D magnetohydrodynamic channel flow by backstepping boundary control. *Syst Control Lett* 57(10):805–812

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Affiliations

Shengze Cai¹ · Shichao Zhou² · Chao Xu¹  · Qi Gao³

✉ Chao Xu
cxu@zju.edu.cn

¹ State Key Laboratory of Industrial Control Technology and the Institute of Cyber-Systems and Control, Zhejiang University, Hangzhou 310027, Zhejiang, China

² School of Mathematical Sciences, Zhejiang University, Hangzhou 310027, Zhejiang, China

³ School of Aeronautics and Astronautics, Zhejiang University, Hangzhou 310027, Zhejiang, China